

download_era5_CDS

May 22, 2026

1 Download ERA5 Data from CDS

This notebook downloads ERA5 reanalysis data from the Climate Data Store (CDS) using the CDS API. It retrieves both surface (single-level) and atmospheric (pressure-level) variables and saves them as NetCDF files.

The workflow is structured to: - Loop over a specified time range - Download data at 6-hour intervals - Organize files into a consistent directory structure

1.1 Imports and Setup

We import standard Python libraries along with the CDS API client.

- `datetime` is used to iterate over time
- `pathlib` is used for file and directory management
- `cdsapi` provides access to ERA5 data

```
[ ]: # Core libraries
import requests
from datetime import datetime, timedelta
from pathlib import Path

# CDS API
import cdsapi
```

1.2 Configuration

This section defines: - The root directory for storing ERA5 data - The start and end dates for retrieval - The CDS API client

Adjust the date range as needed. Data will be downloaded at 6-hour intervals between these dates.

```
[ ]: # Root directory where data will be stored
root_dir = "./era5" # change if needed

# Define time range
start_date = datetime(2024, 12, 1, 0) # modify
end_date = datetime(2024, 12, 31, 18) # modify

# Initialize CDS API client
```

```
c = cdsapi.Client()
```

1.3 Variable Lists

ERA5 variables are grouped into three categories:

1.3.1 Static Variables

These do not change with time (for example, land-sea mask, orography).

1.3.2 Surface Variables

These are single-level atmospheric or land surface quantities (for example, 2m temperature, surface pressure).

Only instantaneous variables are included to avoid mixed output formats.

1.3.3 Atmospheric Variables

These are defined on pressure levels and include 3D fields such as temperature, wind, and humidity.

Pressure levels range from 1 hPa to 1000 hPa.

```
[ ]: # Static variables (optional download)
static_lst = [
    "geopotential",
    "land_sea_mask",
    "slope_of_sub_gridscale_orography",
    "standard_deviation_of_orography"
]

# Surface variables (instantaneous)
surface_lst = [
    "10m_u_component_of_wind",
    "10m_v_component_of_wind",
    "2m_temperature",
    "2m_dewpoint_temperature",
    "mean_sea_level_pressure",
    "sea_surface_temperature",
    "surface_pressure",
    "skin_temperature",
    "total_column_water"
]

# Atmospheric (pressure-level) variables
atmospheric_lst = [
    "temperature",
    "u_component_of_wind",
    "v_component_of_wind",
    "vertical_velocity",
```

```

    "specific_humidity",
    "geopotential"
]

```

1.4 Download Function

This function downloads data for a single timestep.

For each timestamp: - Surface data is saved under: `surface_hourly/inst/YEAR/MONTH/`

- Atmospheric data is saved under: `pressure_hourly/inst/YEAR/MONTH/`

Each file is named using the timestamp: `YYYYMMDD_HHz`

Important: - Year and month are computed inside the function to ensure correct directory structure when looping across months or years. - Errors are caught and printed so the loop can continue if a download fails.

```

[ ]: def download_single_timestep(current_date):

    year = f"Y{current_date.year}"
    month = f"M{current_date.month:02d}"

    print(f"Processing {current_date}")

    # -----
    # Surface data
    # -----
    surf_dir = Path(root_dir) / "surface_hourly" / "inst" / year / month
    surf_dir.mkdir(parents=True, exist_ok=True)

    surf_file = surf_dir / f"era5_surface-inst_allvar_{current_date.
→strftime('%Y%m%d_%H')}.z.nc"

    try:
        c.retrieve(
            "reanalysis-era5-single-levels",
            {
                "product_type": "reanalysis",
                "variable": surface_lst,
                "year": str(current_date.year),
                "month": f"{current_date.month:02d}",
                "day": f"{current_date.day:02d}",
                "time": f"{current_date.hour:02d}:00",
                "format": "netcdf"
            },
            str(surf_file)
        )
    print(f"Saved surface: {surf_file}")

```

```

except Exception as e:
    print(f"Surface failed: {e}")

# -----
# Atmospheric data
# -----
atmos_dir = Path(root_dir) / "pressure_hourly" / "inst" / year / month
atmos_dir.mkdir(parents=True, exist_ok=True)

atmos_file = atmos_dir / f"era5_atmos-inst_allvar_{current_date.
↳strftime('%Y%m%d_%H')}z.nc"

try:
    c.retrieve(
        "reanalysis-era5-pressure-levels",
        {
            "product_type": "reanalysis",
            "variable": atmospheric_lst,
            "year": str(current_date.year),
            "month": f"{current_date.month:02d}",
            "day": f"{current_date.day:02d}",
            "time": f"{current_date.hour:02d}:00",
            "pressure_level": [
                "50", "100", "150", "200", "250", "300", "400", "500",
↳"600", "700", "850", "925", "1000"
            ],
            "format": "netcdf"
        },
        str(atmos_file)
    )
    print(f"Saved atmos: {atmos_file}")
except Exception as e:
    print(f"Atmos failed: {e}")

```

1.5 Time Loop

This loop iterates from the start date to the end date in 6-hour increments.

The 6-hour interval aligns with common synoptic times: - 00Z - 06Z - 12Z - 18Z

For each timestep, the download function is called.

```

[ ]: current_date = start_date

while current_date <= end_date:
    download_single_timestep(current_date)
    current_date += timedelta(hours=6)

```

1.6 Static Data (Optional)

Static variables only need to be downloaded once.

They are saved to: `/static/era5_static-allvar.nc`

This step can be skipped if static data has already been retrieved.

```
[ ]: # Run once if needed

stat_dir = Path(root_dir) / "static"
stat_dir.mkdir(parents=True, exist_ok=True)

stat_file = stat_dir / "era5_static-allvar.nc"

try:
    c.retrieve(
        "reanalysis-era5-single-levels",
        {
            "product_type": "reanalysis",
            "variable": static_lst,
            "year": str(start_date.year),
            "month": f"{start_date.month:02d}",
            "day": f"{start_date.day:02d}",
            "time": f"{start_date.hour:02d}:00",
            "format": "netcdf"
        },
        str(stat_file)
    )
    print(f"Saved static: {stat_file}")
except Exception as e:
    print(f"Static failed: {e}")
```

1.7 Notes and Considerations

- CDS may return ZIP files instead of NetCDF if:
 - Too many variables are requested
 - Mixed parameter types are included (for example, instantaneous and accumulated)
- Keeping variable lists restricted to instantaneous parameters avoids this issue.
- The directory structure is designed to support downstream workflows such as:
 - Machine learning pipelines
 - Batch data loading
 - Time-based indexing
- This modular setup makes it easier to:
 - Parallelize downloads
 - Integrate with PyTorch datasets

- Extend to additional variables or levels